

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Language Models
- N-Grams
- 3.2 Evaluating Language Models: Training and Test Sets
- 3.3 Evaluating Language Models: Perplexity
- 3.4 Sampling sentences from a language model
- 3.5 Generalization and Zeros
- 3.6 Smoothing
- 3.8 Advanced: Kneser-Ney Smoothing

Language Modeling

- ▶ Language modeling involves predicting the probability distribution of words or tokens in a sequence of text. The goal of language modeling is to capture the underlying structure and patterns of natural language, allowing computers to generate coherent and grammatically correct text.
- ▶ There are several approaches to language modeling, including:
 - i) N-gram Models
 - ii) Neural Network Models
 - iii) Transformer Models

Language Modeling

► Tashkent is the capital of -----?

- i) India
- ii) China
- iii) Uzbekistan

Language model applications

- ▶ Spell checking
- ▶ Grammer Checking
- ▶ Machine translation
- ▶ Summarization
- ▶ Question answering
- ▶ Speech recognition

Probabilistic Language Models

- ▶ Assign a probability to a sentence

Application:

- ▶ Machine Translation:

$P(\text{high winds } \text{tonite}) > P(\text{large winds } \text{tonite})$

- ▶ Spell Correction

- ▶ The office is about fifteen **minuets** from my house

- ▶ $P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$

- ▶ Speech Recognition

- ▶ $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

- ▶ + **Summarization**, question-answering,

Probability of sentence

- ▶ Grammer correction
 - ▶ I go to school
 - ▶ I going to school
- ▶ Probability score: I go to school > I going to school
- ▶ Correct: go to school, Wrong: going to school

Probability of sentence or words

- ▶ Compute the probability of a sentence or sequence of words:
 $\Rightarrow P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$
- ▶ Probability of an upcoming word:
 $\Rightarrow P(w_5 | w_1, w_2, w_3, w_4)$
- ▶ $P(\text{Uzbekistan} | \text{Tashkent, is, the, capital, of})$
- ▶ A model that computes either of these :
 $P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a language model.

How to compute $P(W)$

- ▶ How to compute this joint probability:
 - ▶ $P(\text{its, water, is, so, transparent, that})$
- ▶ Intuition: let's rely on the Chain Rule of Probability

$$P(A,B) = p(A|B) p(B)$$

We can extend this for three variables:

$$P(A,B,C) = P(A|B,C) P(B,C) = P(A|B,C) P(B|C) P(C)$$

and in general to n variables:

$$P(A_1, A_2, \dots, A_n) = P(A_1|A_2, \dots, A_n) P(A_2|A_3, \dots, A_n) \\ P(A_{n-1}|A_n) P(A_n)$$

In general we refer to this as the *chain rule*

the joint probability of all the random variables can be calculated by multiplying the probability of each variable conditioned on all the previous variables

Chain Rule of Probability

- ▶ Conditional probabilities

$$\Rightarrow P(B|A) = P(A,B) / P(A)$$

$$\text{Rewriting : } P(A,B) = P(A)P(B|A)$$

$$\text{More variables: } P(A,B,C,D) = P(A) P(B|A) P(C|A, B) P(D|A,B,C)$$

The chain rule in general

$$\Rightarrow P(x_1, x_2, x_3, \dots, x_n) = P(x_1) P(x_2|x_1) P(x_3|x_1,x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

Chain Rule of Probability

► Chain rule : $P(A,B,C,D) = P(A) P(B|A) P(C|A,B) P(D|A,B,C)$

► Example

= $P(\text{Tashkent is the capital of Uzbekistan})$

= $P(\text{Tashkent}) \times P(\text{is} | \text{Tashkent}) \times P(\text{the} | \text{Tashkent, is}) \times P(\text{capital} | \text{Tashkent, is the}) \times P(\text{of} | \text{Tashkent, is, the, capital}) \times P(\text{Uzbekistan} | \text{Tashkent, is, the, capital, of})$

Chain Rule of Probability

► Example

= P (Tashkent is the capital of Uzbekistan)

= P(Tashkent) X P(is | Tashkent)X P(the| Tashkent, is) x P(capital | Tashkent, is, the) X P(of | Tashkent, is, the, capital) x P(Uzbekistan| Tashkent, is ,the, capital, of)

Calculation

= P(Uzbekistan| Tashkent, is ,the, capital, of)

= count (Tashkent is the capital of Uzbekistan) / count (Tashkent is the capital of)

The Chain Rule applied to compute joint probability of words in sentence

$$P(w_1 w_2 \square w_n) = \prod_i P(w_i \mid w_1 w_2 \square w_{i-1})$$

$P(\text{"its water is so transparent"}) =$

$$\begin{aligned} &P(\text{its}) \times P(\text{water} \mid \text{its}) \times P(\text{is} \mid \text{its water}) \\ &\times P(\text{so} \mid \text{its water is}) \times P(\text{transparent} \mid \text{its water is so}) \end{aligned}$$

How to estimate these probabilities

- ▶ Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{\text{Count}(\text{its water is so transparent that the})}{\text{Count}(\text{its water is so transparent that})}$$

- ▶ No! Too many possible sentences!
- ▶ We'll never see enough data for estimating these

Markov Assumption

- ▶ Simplifying assumption
 - = $P(\text{Uzbekistan} \mid \text{Tashkent, is, the, capital, of})$
 - = $P(\text{Uzbekistan} \mid \text{of})$
 - = $P(\text{Uzbekistan} \mid \text{capital of})$



Andrei Markov

The assumption that the probability of a word depends only on the previous word is called Markov assumption

Simplest case: Unigram model

$$P(w_1 w_2 \square \dots w_n) \square \square \square P(w_i)$$

Some automatically generated sentences from a unigram model

fifth, an, of, futures, the, an, incorporated, a,
a, the, inflation, most, dollars, quarter, in, is,
mass

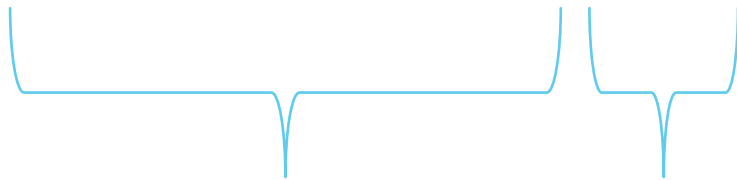
thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the

Bigram model

- ▶ Condition on previous word

- ▶ Please bring me a glass of **water**.



History

Word prediction

Estimating bigram probabilities

- ▶ The Maximum Likelihood Estimate

$$P(w_i | w_{i-1}) = \frac{\textit{count}(w_{i-1}, w_i)}{\textit{count}(w_{i-1})}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Bigram model

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Estimated bigram probabilities

- ▶ $P(<s> \text{ I want English food } </s>) = P(\text{I} | <s>) \times P(\text{want} | \text{I}) \times P(\text{English} | \text{want}) \times p(\text{food} | \text{english}) \times P(</s> | \text{food}) = 0.000031$

- ▶ Given that

$$P(\text{I} | <s>) = 0.25$$

$$P(\text{want} | \text{I}) = 0.33$$

$$P(\text{English} | \text{want}) = 0.0011$$

$$p(\text{food} | \text{english}) = 0.5$$

$$P(</s> | \text{food}) = 0.68$$

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

N-gram models

- ▶ We can extend to trigrams, 4-grams, 5-grams
- ▶ In general this is an insufficient model of language
 - ▶ because language has **long-distance dependencies**:

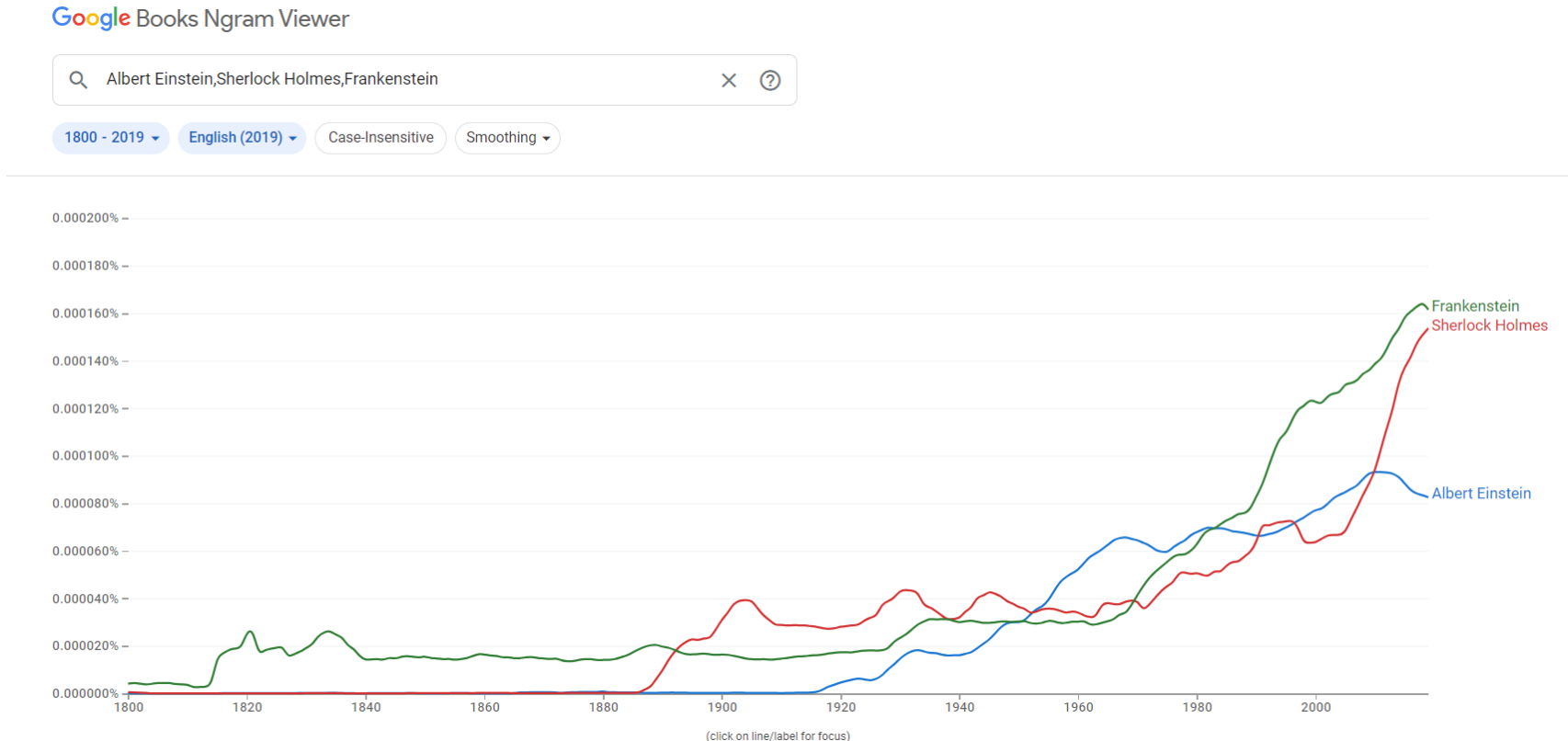
“The computer which I had just put into the machine room on the fifth floor crashed.”

- ▶ But we can often get away with N-gram models

N-gram models

- ▶ An n-gram is a collection of n successive items in a text document that may include words, numbers, symbols, and punctuation. N-gram models are useful in many text analytics applications where sequences of words are relevant, such as in sentiment analysis, text classification, and text generation.
- ▶ In deep learning , Language models used higher gram model to train the dataset.

N-gram models



Google Ngram Viewer displays user-selected words or phrases (ngrams) in a graph that shows how those phrases have occurred in a corpus. Google Ngram Viewer's corpus is made up of the scanned books available in Google Book

Once the language model is built, it can then be used with machine learning algorithms to build predictive models for text analytics applications

Google N-Gram Release, August 2006

AUG

3

All Our N-gram are Belong to You

Posted by Alex Franz and Thorsten Brants, Google Machine Translation Team

Here at Google Research we have been using word [n-gram models](#) for a variety of R&D projects,

...

That's why we decided to share this enormous dataset with everyone. We processed 1,024,908,267,229 words of running text and are publishing the counts for all 1,176,470,663 five-word sequences that appear at least 40 times. There are 13,588,391 unique words, after discarding words that appear less than 200 times.

Evaluating Language Models: Training and Test Sets

- ▶ “Extrinsic Evaluation” a method of assessing the quality of a system by evaluating its performance on downstream tasks

To compare models A and B

1. Put each model in a real task
 - Machine Translation, speech recognition, etc.
2. Run the task, get a score for A and for B
 - How many words translated correctly
 - How many words transcribed correctly
3. Compare accuracy for A and B

Intrinsic evaluation

- ▶ Extrinsic evaluation not always possible
 - Expensive, time-consuming
 - Doesn't always generalize to other applications
- ▶ Intrinsic evaluation: **perplexity**
 - Directly measures language model performance at predicting words.
 - Doesn't necessarily correspond with real application performance
 - But gives us a single general metric for language models
 - Useful for large language models (LLMs) as well as n-grams

Training sets and test sets

We train parameters of our model on a **training set**.

We test the model's performance on data we haven't seen.

- ▶ A **test set** is an unseen dataset; different from training set.
 - ▶ Intuition: we want to measure generalization to unseen data
- ▶ An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.

Perplexity

- ▶ Perplexity is the standard metric for measuring quality of a language model.
- ▶ The inverse probability of test set, normalized by the number of words.

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Chain rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

Bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the maximizing probability

Perplexity

- ▶ Calculate perplexity of a sentence

Task of recognizing the digit in English

=> A sentence consist of random digits

=> Each digit probability is $p = 1/10$

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}\right)^{-\frac{N}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10 \end{aligned}$$

Minimizing perplexity is the maximizing probability

Perplexity

Lower perplexity = better model

Training 38 million words, test 1.5 million words, from Wall Street Journal

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Choosing training and test sets

- If we're building an LM for a specific task
 - The test set should reflect the task language we want to use the model for
- If we're building a general-purpose model
 - We'll need lots of different kinds of training data
 - We don't want the training set or the test set to be just from one domain or author or language.

Training on the test set

We can't allow test sentences into the training set

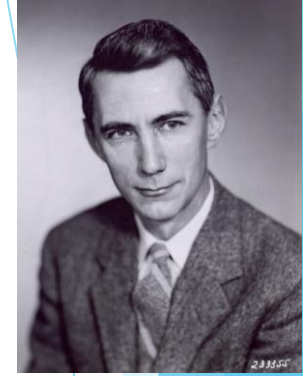
- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called “Training on the test set”

Dev sets

- If we test on the test set many times we might implicitly tune to its characteristics
 - Noticing which changes make the model better.
- So we run on the test set only once, or a few times
- That means we need a third dataset:
 - A **development test set** or, **devset**.
 - We test our LM on the devset until the very end
 - And then test our LM on the **test set** once

Sampling and Generalization



- The Shannon (1948) Visualization Method Sample words from an LM

- ▶ Unigram:

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE
HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE
HAD BE THESE.

- ▶ Bigram:

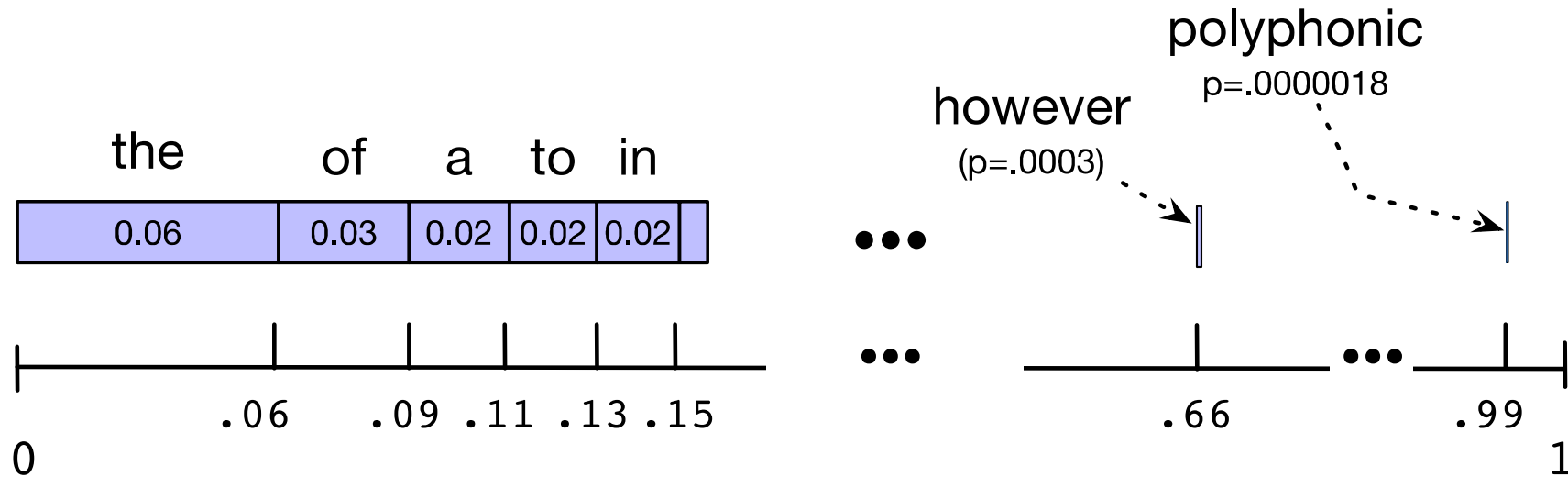
THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE
CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS
THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED.

How Shannon sampled those words in 1948



Open a book at random and select a letter at random on the page. This letter is recorded. The book is then opened to another page and one reads until this letter is encountered. The succeeding letter is then recorded. Turning to another page this second letter is searched for and the succeeding letter recorded, etc."

Sampling a word from a distribution



Visualizing Bigrams the Shannon Way

Choose a random bigram ($\langle s \rangle$, w)

according to its probability $p(w|\langle s \rangle)$

Now choose a random bigram (w, x) according to its probability $p(x|w)$

And so on until we choose $\langle /s \rangle$

Then string the words together

$\langle s \rangle$ I
I want
want to
to eat
eat Chinese
Chinese food
food $\langle /s \rangle$
I want to eat Chinese food

There are other sampling methods

Used for neural language models

Many of them avoid generating words from the very unlikely tail of the distribution

We'll discuss when we get to neural LM decoding:

- ▶ Temperature sampling
- ▶ Top-k sampling
- ▶ Top-p sampling

Approximating Shakespeare

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

Corpus

- ▶ A corpus refers to a large and structured set of machine-readable texts
- ▶ Corpus texts are collections from
 - Books
 - Article
 - Websites
 - Conversation
 - Social media
 - Audio

Shakespeare as corpus

N=884,647 tokens, V=29,066

Shakespeare produced 300,000 bigram types out of $V^2 = 844$ million possible bigrams.

- ▶ So 99.96% of the possible bigrams were never seen (have zero entries in the table)
- ▶ That sparsity is even worse for 4-grams, explaining why our sampling generated actual Shakespeare

Shakespeare as Corpus

- ▶ Total works:43
- ▶ Words: 884,421
- ▶ Unique word forms:28,829,
- ▶ Word occurring only once: 12,493

The Wall Street Journal is not Shakespeare

1
gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2
gram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Choosing training data

If task-specific, use a training corpus that has a similar genre to your task.

- If legal or medical, need lots of special-purpose documents

Make sure to cover different kinds of dialects and speaker/authors.

- Example: *African-American Vernacular English (AAVE)*
 - One of many varieties that can be used by African Americans and others
 - Can include the auxiliary verb **finna** that marks immediate future tense:
 - "My phone finna die"

Finna → going to

Why do we need corpus in NLP

- ▶ Training machine learning models

- ✓ Sentiment analysis, speech recognition, machine translation

Language understanding

- ✓ Grammar, vocabulary

Rule based system

- ✓ Part-of-speech(POS) tagging, named entity recognition

Statistical analysis

- ✓ Examine word frequency distribution, statistical features

Domain specific Knowledge

- ✓ Legal documents, medical documents, chatbots

Training Corpus

- ▶ Building a question answering system, we need a training corpus of question answer
- ▶ To build translating legal documents, we need a training corpus of legal documents
- ▶ N gram only works well for word prediction if the test corpus looks like the training corpus.

The perils of overfitting

N-grams only work well for word prediction if the test corpus looks like the training corpus

- But even when we try to pick a good training corpus, the test set will surprise us!
- We need to train robust models that generalize!

One kind of generalization: **Zeros**

- Things that don't ever occur in the training set
 - But occur in the test set

Zeros

► Training set:

.. denied the allegations
... denied the reports
... denied the claims
.... denied the request

► $P(\text{"offer"} \mid \text{denied the}) = 0$

Test set:

... denied the offer
.... denied the load

Zero probability bigrams

- ▶ Bigram with zero probability
 - Assign 0 probability to the test set

It is not possible to calculate perplexity

- Divide by zero

Bigrams with zero probability

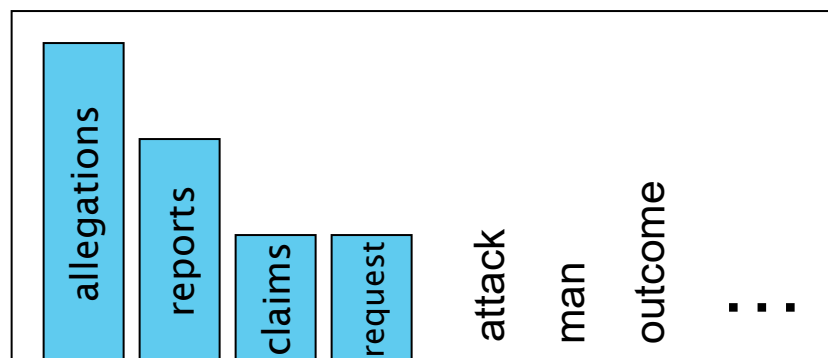
- ▶ Will hurt our performance for texts where those words appear!
- ▶ And mean that we will assign 0 probability to the test set!

And hence we cannot compute perplexity (can't divide by 0)!

Smoothing: Add-one (Laplace) smoothing

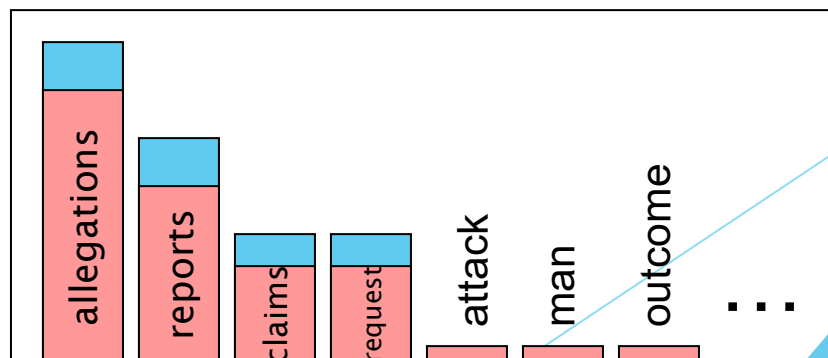
- ▶ When we have sparse statistics:

$P(w \mid \text{denied the})$
3 allegations
2 reports
1 claims
1 request
1 attack
1 man
1 outcome
7 total



- ▶ Steal probability mass to generalize better

$P(w \mid \text{denied the})$
2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



Add-one estimation

- ▶ Also called Laplace smoothing
- ▶ Pretend we saw each word one more time than we did
- ▶ Just add one to all the counts!

- ▶ MLE estimate:

$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- ▶ Add-1 estimate:

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

Maximum Likelihood Estimates

- ▶ The maximum likelihood estimate
 - ▶ of some parameter of a model M from a training set T
 - ▶ maximizes the likelihood of the training set T given the model M
- ▶ Suppose the word “bagel” occurs 400 times in a corpus of a million words
- ▶ What is the probability that a random word from some other text will be “bagel”?
- ▶ MLE estimate is $400/1,000,000 = .0004$
- ▶ This may be a bad estimate for some other corpus
 - ▶ But it is the **estimate** that makes it **most likely** that “bagel” will occur 400 times in a million word corpus.

Example corpus Berkely restaurant project

- ▶ Can you tell me about a good Cantonese restaurant close by.
- ▶ Mid priced thai food is what I am looking for
- ▶ Tell me about chez panisse
- ▶ Can you give me a listing of the kinds of food that are available
- ▶ I am looking for a good place to eat breakfast
- ▶ When is caffe Venezia open during the day.

Berkeley Restaurant Corpus: Laplace smoothed bigram counts

Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Backoff and Interpolation

- ▶ Sometimes it helps to use **less** context
 - ▶ Condition on less context for contexts you haven't learned much about
- ▶ **Backoff:**
 - ▶ use trigram if you have good evidence,
 - ▶ otherwise bigram, otherwise unigram
- ▶ **Interpolation:**
 - ▶ mix unigram, bigram, trigram
- ▶ Interpolation works better

How to set the lambdas

- ▶ Use a **held-out** corpus



- ▶ Choose λ s to maximize the probability of held-out data:
 - ▶ Fix the N-gram probabilities (on the training data)
 - ▶ Then search for λ s that give largest probability to held-out set:

$$\log P(w_1 \dots w_n \mid M(\lambda_1 \dots \lambda_k)) = \sum_i \log P_{M(\lambda_1 \dots \lambda_k)}(w_i \mid w_{i-1})$$

Held-out data refers to a portion of historical, labeled data

Unknown Words

- ▶ If we know the words
 - Vocabulary V is fixed
 - Closed vocabulary task
- ▶ If we don't know the word
 - Out of vocabulary = OOV words
 - Open vocabulary task

- ▶ What we do in this situation

Create an unknown word token <UNK>

Create a fixed lexicon L of size V

In normalization phase, any word not in L changed to <UNK>

We train it's probabilities like a normal word

At decoding time, use UNK probabilities for any word not in training

Huge web-scale n-grams

- ▶ How to deal with, e.g., Google N-gram corpus
- ▶ Pruning
 - ▶ Only store N-grams with count > threshold.
 - ▶ Remove singletons of higher-order n-grams
 - ▶ Entropy-based pruning
- ▶ Efficiency
 - ▶ Efficient data structures like tries
 - ▶ Bloom filters: approximate language models
 - ▶ Store words as indexes, not strings
 - ▶ Use Huffman coding to fit large numbers of words into two bytes
 - ▶ Quantize probabilities (4-8 bits instead of 8-byte float)

“Stupid backoff” (Brants *et al.* 2007)

N-gram Smoothing Summary

- ▶ Add-1 smoothing:
 - ▶ OK for text categorization, not for language modeling
- ▶ The most commonly used method:
 - ▶ Extended Interpolated Kneser-Ney smoothing
- ▶ For very large N-grams like the Web:
 - ▶ Stupid backoff

Advanced Language Modeling

- ▶ Discriminative models:

- ▶ choose n-gram weights to improve a task, not to fit the training set

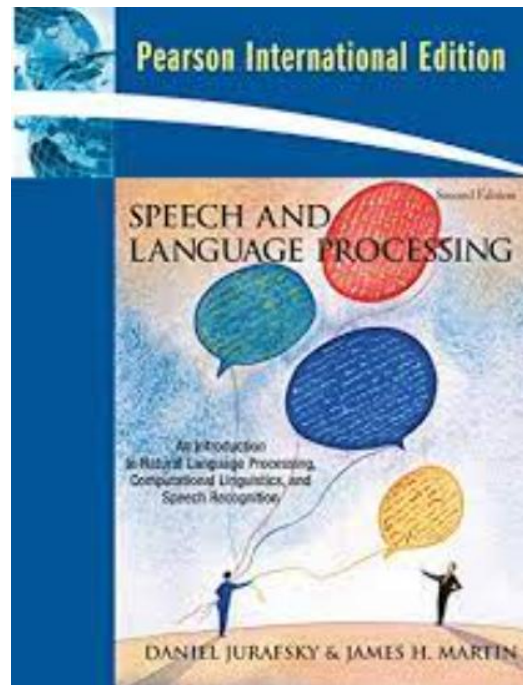
- ▶ Parsing-based models

- ▶ Caching Models

- ▶ Recently used words are more likely to appear

$$P_{CACHE}(w | history) = \lambda P(w_i | w_{i-2} w_{i-1}) + (1 - \lambda) \frac{c(w \square history)}{|history|}$$

Reference



Chapter 3

Question



Thank you

